

선형 회귀
L O

- 회귀의 개념을 이해한다.
- 경사 하강법을 이해한다.
- 과잉 적합과 과소 적합을 이해한다.
- 파이썬과 sklearn을 이용하여 회귀를 구현해본다.



회귀(regression)와 분류(classification)

3

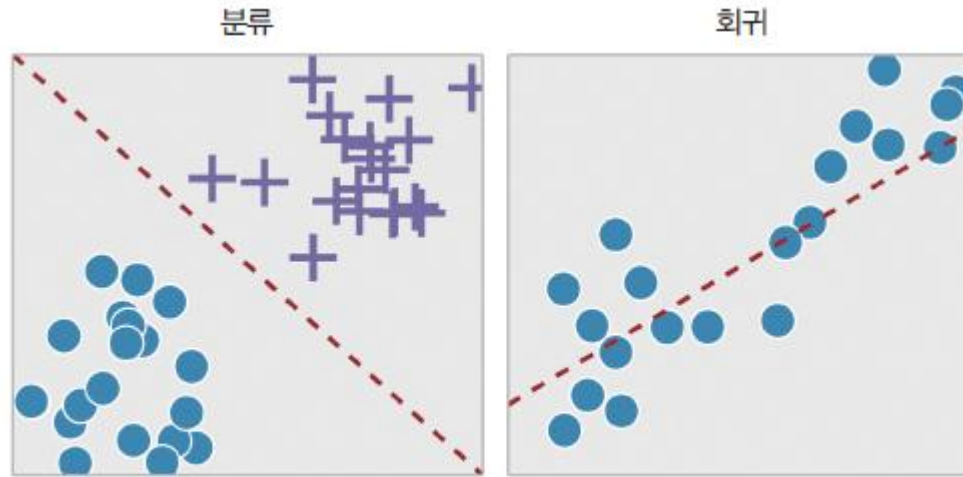


그림 4-1 회귀와 분류



선형 회귀

4

- 회귀란 일반적으로 데이터들을 2차원 공간에 찍은 후에 이들 데이터들을 가장 잘 설명하는 직선이나 곡선을 찾는 문제라고 할 수 있다.
- $y = f(x)$ 에서 출력 y 가 실수이고 입력 x 도 실수일 때 함수 $f(x)$ 를 예측하는 것이 회귀이다.
- 가장 잘 설명하는 직선을 찾는 것을 선형회귀라 할 수 있다.

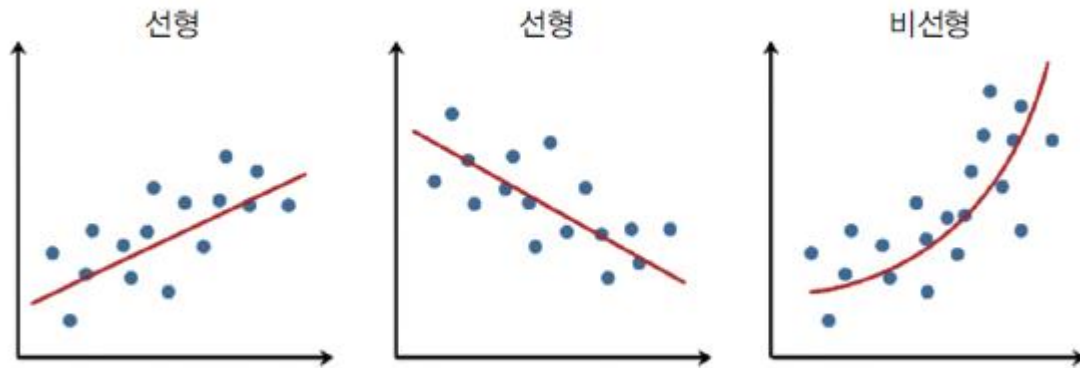


그림 4-2 회귀의 종류



선형 회귀의 예

5

- 부모의 키와 자녀의 키의 관계 조사
- 면적에 따른 주택의 가격
- 연령에 따른 실업율 예측
- 공부 시간과 학점 과의 관계
- CPU 속도와 프로그램 실행 시간 예측



선형 회귀 소개

6

- 선형 회귀는 입력 데이터를 가장 잘 설명하는 기울기와 절편값을 찾는 문제이다
- 선형 회귀의 기본식: $f(x) = Wx + b$
 - 기울기->가중치(Weights, 독립변수가 여러 개일때를 생각하면 편함)
 - 절편->바이어스 (Bias, W_0 로 표기하기도 함)



선형 회귀 예제

7

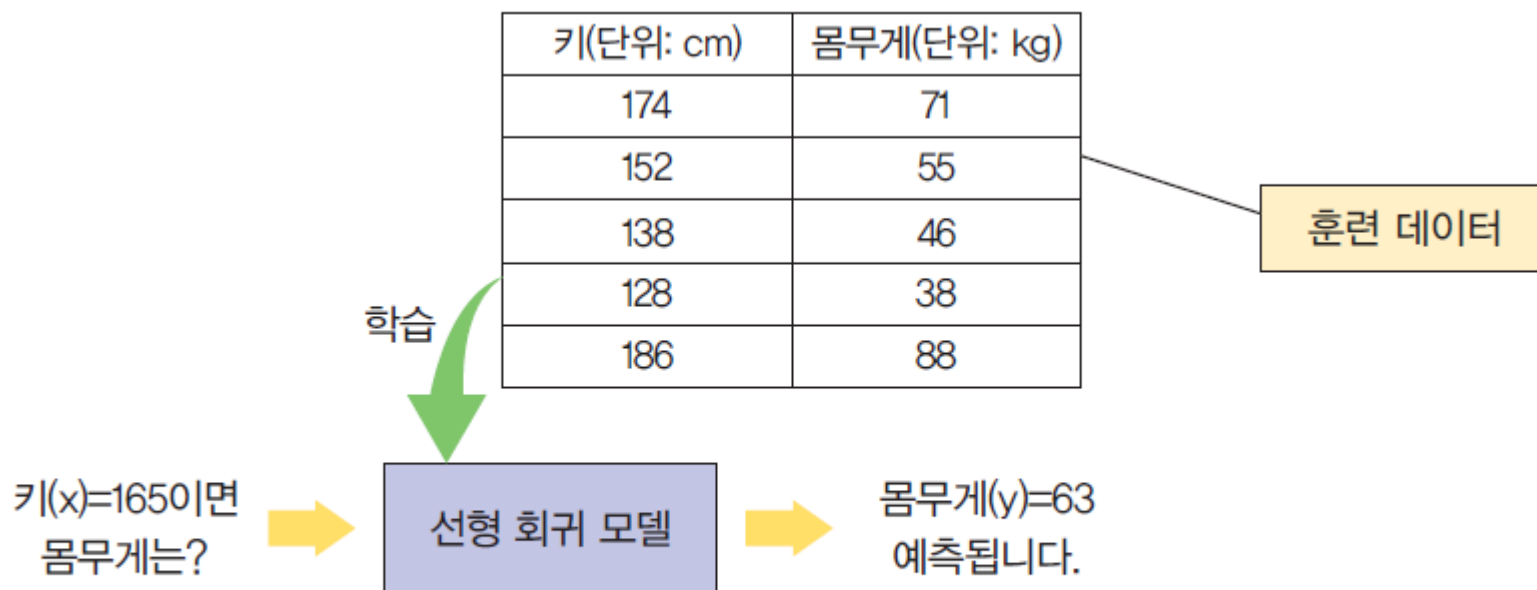


그림 4-3 선형 회귀의 예제



선형 회귀의 종류

8

- 단순 선형 회귀: 단순 선형 회귀는 독립 변수(x)가 하나인 선형 회귀이다.

$$f(x) = wx + b$$

- 다중 선형 회귀: 독립 변수(x)가 여러 개인 경우

$$f(x, y, z) = w_0 + w_1x + w_2y + w_3z$$

$$\text{매출} = w_0 + w_1 \times \text{인터넷 광고} + w_2 \times \text{신문 광고} + w_3 \times \text{TV광고}$$

$$f(x_1, x_2, x_3) = w_0 + w_1x_1 + w_2x_2 + w_3x_3 \text{ 가 좀 더 일반적인 표현}$$

w_1, w_2, w_3 를 가중치로 부르는 이유?

$\Rightarrow w_1$ 이 0이라면 어떤 일이 생길까?



선형 회귀의 원리

9

x	y
1	2
2	5
3	6

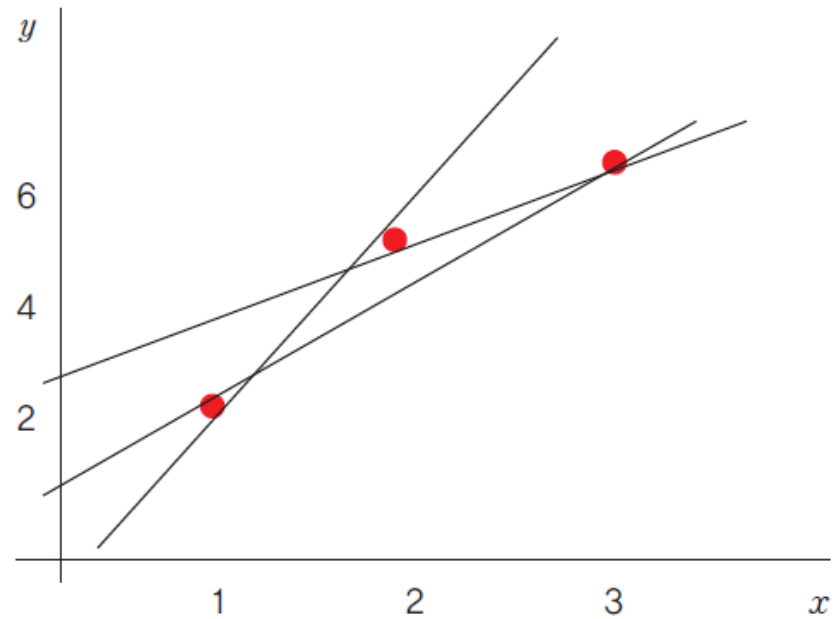


그림 4-4 데이터와 직선



직선과 데이터의 거리

10

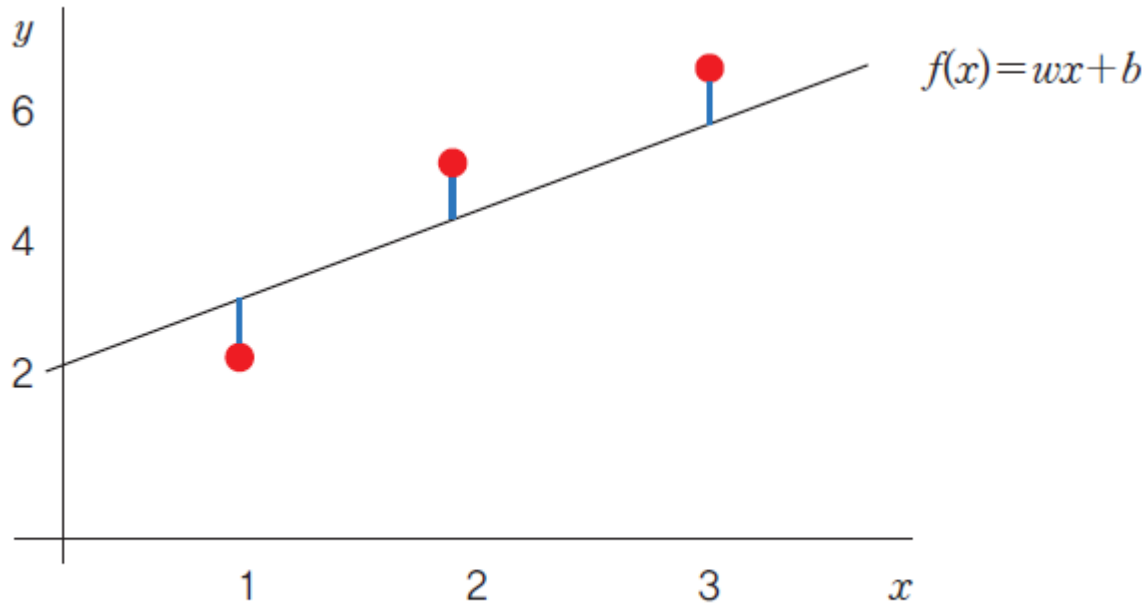


그림 4-5 데이터와 직선 간의 거리 = $|y - f(x)|$



- 직선(예측치)과 데이터(실제값) 사이의 간격을 제공하여 합한 값(MSE=Mean Squared Error)을 손실 함수(loss function) 또는 비용 함수(cost function)로 정의함.
- 직선(예측치)과 데이터(실제값) 사이의 간격의 절대값은 MAE(Mean Absolute Error)로 불리며, 이 또한 손실함수 또는 비용함수로 사용될 수 있다.
- 손실(loss) 또는 비용(cost)가 작을 수록 성능이 좋은 것임

$$Loss = \frac{1}{3} ((f(x_1) - y_1)^2 + (f(x_2) - y_2)^2 + (f(x_3) - y_3)^2)$$



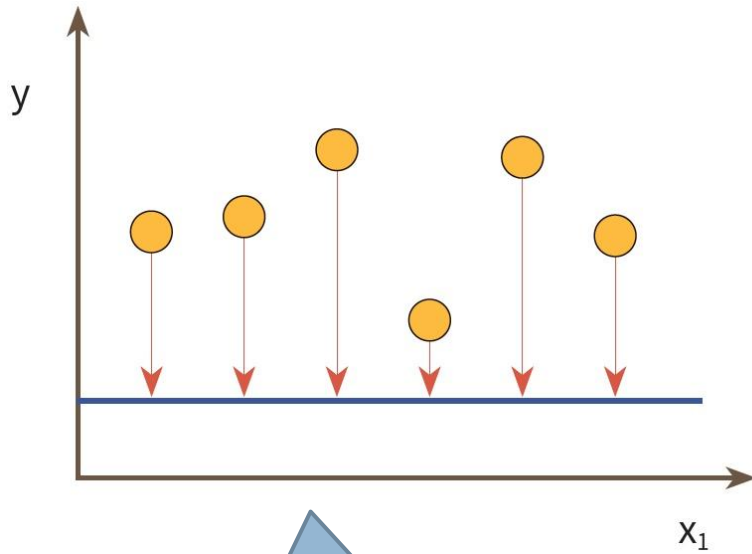
$$Loss(W, b) = \frac{1}{n} \sum_{i=1}^n (f(x_i) - y_i)^2 = \frac{1}{n} \sum_{i=1}^n ((Wx_i + b) - y_i)^2$$



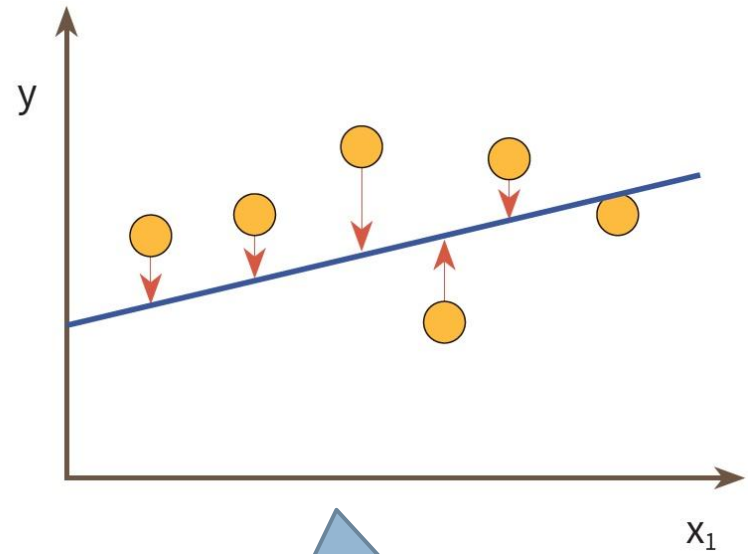
$$\underset{W, b}{\operatorname{argmin}} Loss(W, b)$$



12



순실이 큰 경우



순실이 작은 경우

WIN!!!



선형 회귀에서 손실 함수 최소화 방법

13

- 분석적인 방법: 독립 변수와 종속 변수가 각각 하나인 선형 회귀
(최소제곱법이란 방법을 통한 공식)
독립 변수가 여러 개이면 적용 안됨!!!
(여기서 n 은 변수의 개수가 아니라, 데이터의 개수임. 혼동 금지)

$$w = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad b = \bar{y} - w\bar{x}$$

- 경사 하강법 (gradient descent method):
 - 경사 하강법은 손실 함수가 어떤 형태이러도, 또 매개(독립) 변수가 아무리 많아도 적용할 수 있는 일반적인 방법이다.
 - 점진적인 학습이 가능하다



경사하강법

14

경사(Gradient) = 기울기 = 변화량

$$\text{기울기} = \{ f(w+\Delta) - f(w) \} / \Delta \quad (\Delta: \text{작은 양})$$

함수를 미분하면 기울기 함수가 되며, w 에서의 기울기는 $f'(w)$ 로도 표기할 수 있음

경사하강법이란 경사(Gradient, 기울기)의 크기 (=기울기의 절댓값)가 작아지는 방향으로 w 를 이동 (업데이트) 하는 것임 (경사의 크기가 작아지는 방향이 함수의 값이 작아지는 방향과 일치하는 것이 포인트)

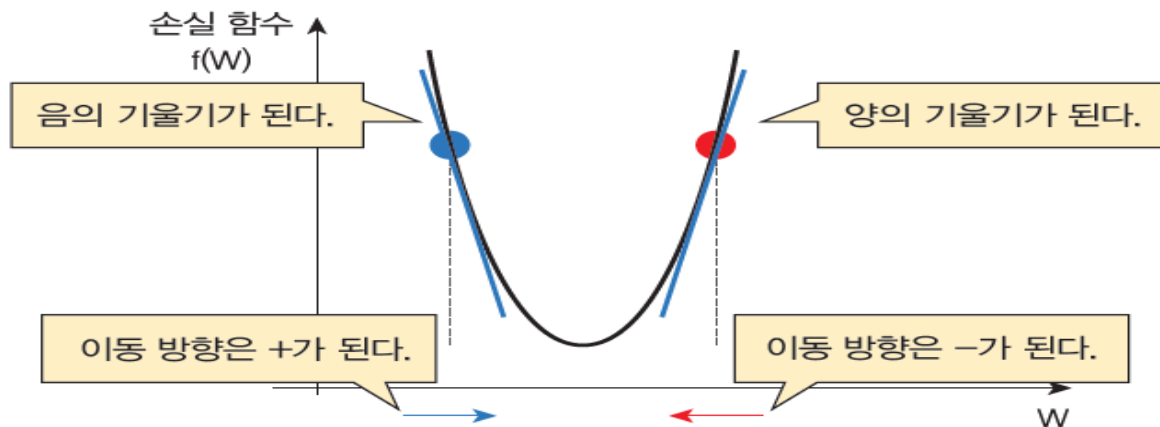


그림 4-7 경사 하강법의 이해



경사하강법

15



이것은 마치 산에서 내려오는 것과 유사합니다. 현재 위치에서 산의 기울기를 계산하여서 기울기의 반대 방향으로 이동하면 산에서 내려오게 됩니다.



$$\text{기울기의 크기} = |\text{기울기값}|$$

기울기의 크기가 **하**강하는 곳으로 이동하자



- 미분이란 주어진 함수의 기울기 함수를 구하는 것임
- $f(x)$ 를 x 에 대해서 미분하면 $f'(x)$ 인 기울기 함수가 구해진다.

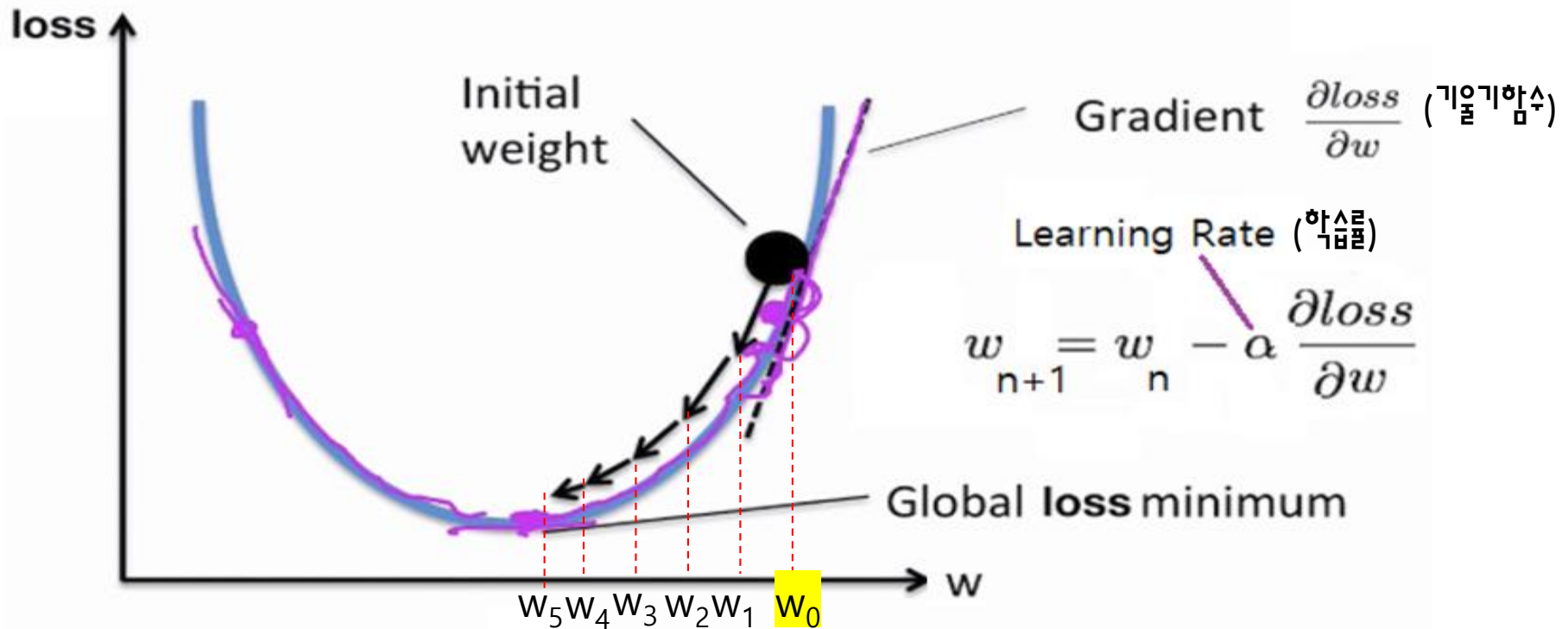
$f(x) = (3x+1)^2$ 를 x 에 대해 미분해보세요

1. $\frac{d}{dx}(c) = 0$
2. $\frac{d}{dx}[cf(x)] = cf'(x)$
3. $\frac{d}{dx}[f(x)+g(x)] = f'(x)+g'(x)$
4. $\frac{d}{dx}[f(x)-g(x)] = f'(x)-g'(x)$
5. $\frac{d}{dx}[f(x)g(x)] = f(x)g'(x)+g(x)f'(x)$ 곱셈공식
6. $\frac{d}{dx}\left[\frac{f(x)}{g(x)}\right] = \frac{g(x)f'(x)-f(x)g'(x)}{[g(x)]^2}$ 나눗셈공식
7. $\frac{d}{dx}f(g(x)) = f'(g(x))g'(x)$ 연쇄법칙(체인룰)
8. $\frac{d}{dx}(x^n) = nx^{n-1}$



경사하강법 공식

17



- w_0 가 시작값(Initial weight)이며, 보통 랜덤하게 정한다.
- 학습률에 따라 업데이트 되는 양이 조절된다.
- 이론적으로는 loss가 더 이상 줄어들지 않을 때까지 반복 한다.



경사하강법 공식

18

$$\text{Loss}(w) = (w-2)^2, w_0 = 8, \alpha = 0.1 \quad w_2 = ?$$

$\Rightarrow$ 파이

$$\text{Loss}'(w) = \partial \text{Loss} / \partial w = 2(w-2)$$

$$\begin{aligned} w_1 &= w_0 - (0.1) \times (\text{Loss}'(w_0)) \\ &= 8 - (0.1) \times (\text{Loss}'(8)) = 8 - 1.2 = 6.8 \end{aligned}$$

$$\begin{aligned} w_2 &= w_1 - (0.1) \times (\text{Loss}'(w_1)) \\ &= ? \end{aligned}$$

$$w_{n+1} = w_n - \alpha \frac{\partial \text{loss}}{\partial w}$$

Learning Rate



- 편미분이란 여러 변수에 대한 함수를 미분할 때, 여러 변수들 중 하나에 대해서 미분하고, 나머지는 상수로 취급하는 것을 의미한다.

$$f(x, y) = x^2 + xy + y^2$$



x 에 대해 편미분

y 에 대해 편미분 결과?

$$\frac{\partial f}{\partial x}(x, y) = 2x + y$$

$f(x, y, w, b) = (wx + b - y)^2$ 를 w 와 b 에 대해서 편미분하세요



학습률(=학습되는 속도)

20

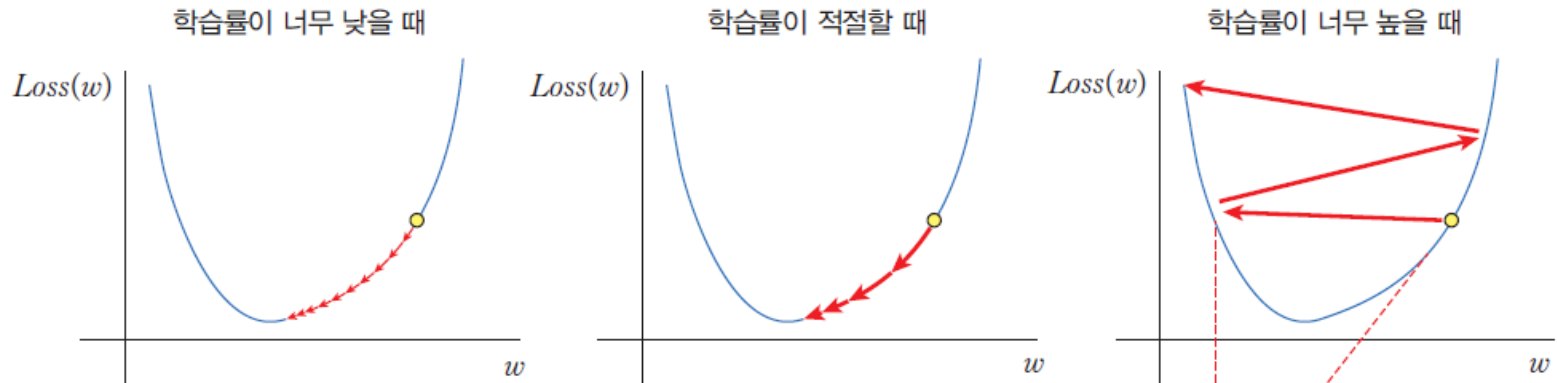


그림 4-9 학습률

Learning Rate

$$w_{n+1} = w_n - \alpha \frac{\partial loss}{\partial w}$$

학습률이 크면 업데이트되는 양이 커지고,
 학습률이 작으면 업데이트되는 양이 작아진다.
 즉, 학습률은 학습속도를 조절할 수 있다.



지역최소값 문제

21

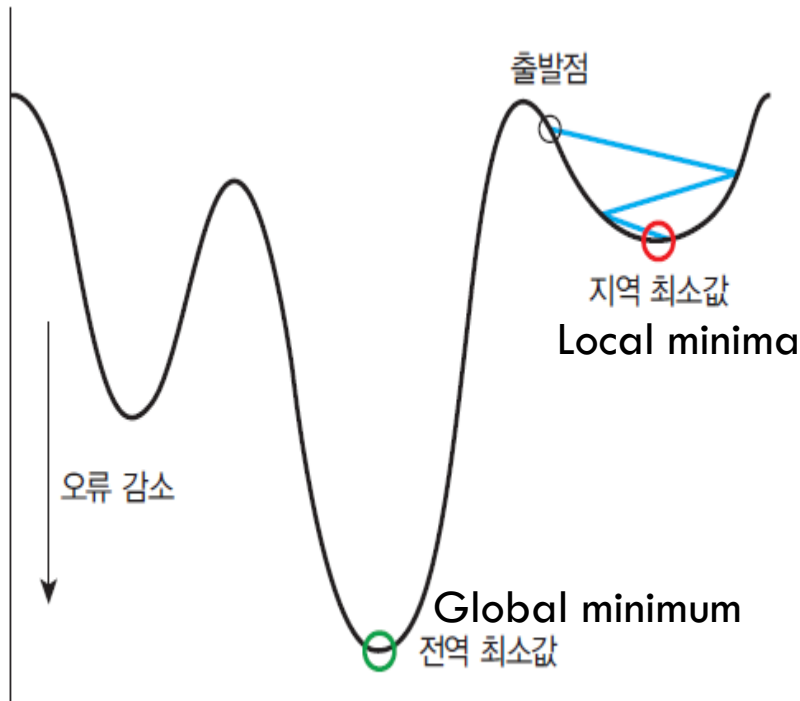


그림 4-10 지역 최소값과 전역 최소값

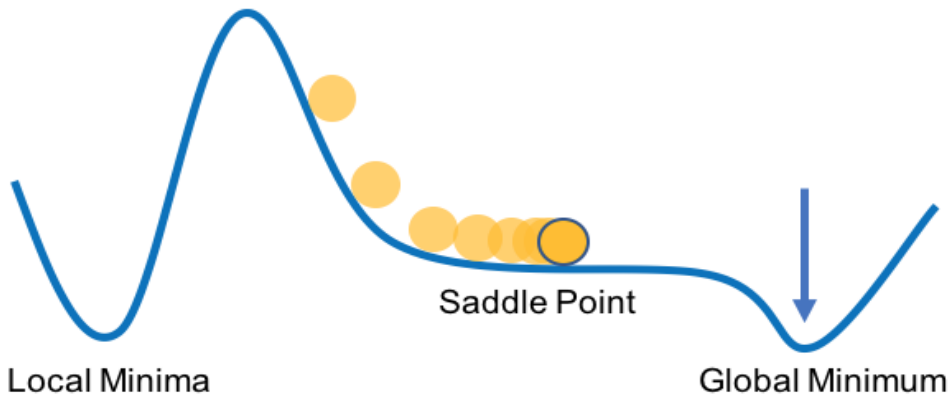
- 지역 최소값을 벗어나기 위한 방법은?

- (1) 초기값을 랜덤하게 정하여, 경사하강법을 여러 번 적용한다.
- (2) 학습률을 초반에 크게 잡았다가 진행될수록 학습률을 작게 한다.



지역최소화 문제

22



왼쪽 그림의 경우 경사하강법이 제대로 적용이 안 되는 이유가 뭘까?

$$w_{n+1} = w_n - \alpha \frac{\partial loss}{\partial w}$$

Learning Rate



선형 회귀에서 경사하강법

23

$$Loss(W, b) = \frac{1}{n} \sum_{i=1}^n ((Wx_i + b) - y_i)^2$$

n 은 데이터의 개수임,
Loss가 평균 오차 (MSE)로 정의됨

$$\frac{\partial Loss(W, b)}{\partial W} = \frac{1}{n} \sum_{i=1}^n 2((Wx_i + b) - y_i)(x_i) = \frac{2}{n} \sum_{i=1}^n x_i((Wx_i + b) - y_i)$$

$$\frac{\partial Loss(W, b)}{\partial b} = \frac{2}{n} \sum_{i=1}^n ((Wx_i + b) - y_i)$$

$$W = W - 0.01 * \frac{\partial Loss}{\partial W}$$

$$b = b - 0.01 * \frac{\partial Loss}{\partial b}$$

0.01은 설정된 학습률, 사용자가 설정함
학습률이 클 수록 업데이트되는 양이 커짐



선형 회귀에서 경사하강법

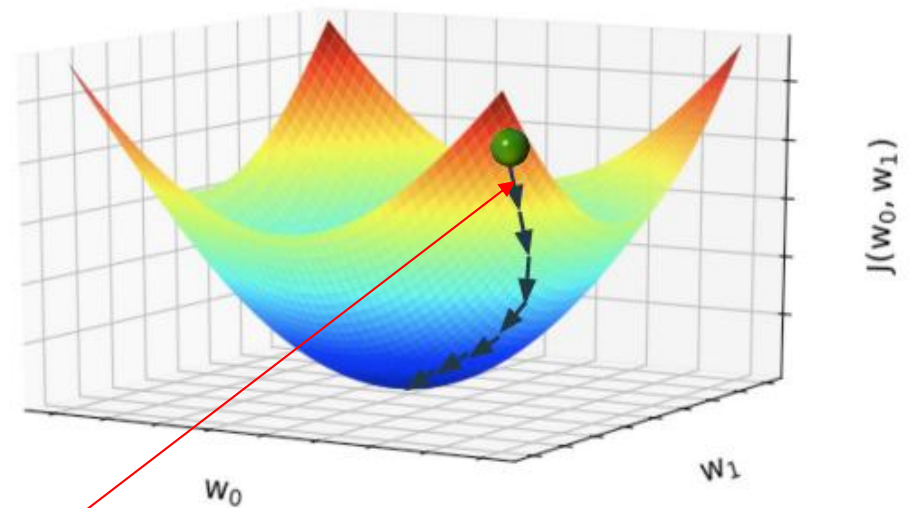
24

- 일반화를 위해, b 대신 w_0 를 w 대신 w_1 을 사용
- $J(w_0, w_1)$ 은 loss function

$$w_0 = w_0 - \alpha \frac{\partial}{\partial w_0} J(w_0, w_1)$$

$$w_1 = w_1 - \alpha \frac{\partial}{\partial w_1} J(w_0, w_1)$$

우변에 적용되는 값은 현재의 값이고,
그런 현재의 값으로 계산된 결과가 좌변인 것임



$$\left(-\alpha \frac{\partial}{\partial w_0} J(w_0, w_1), -\alpha \frac{\partial}{\partial w_1} J(w_0, w_1) \right)$$



경사 하강법 구현

25

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([0.0, 1.0, 2.0])
y = np.array([3.0, 3.5, 5.5])

W = 0    # 기울기, 보통은 랜덤하게 초기화
b = 0    # 절편, 보통은 랜덤하게 초기화

lrate = 0.01 # 학습률
epochs = 1000 # 반복 횟수

n = float(len(X)) # 입력 데이터의 개수

# 경사 하강법
for i in range(epochs): #
    y_pred = W*X + b # 예측값
    dW = (2/n) * sum(X * (y_pred-y))
    db = (2/n) * sum(y_pred-y)
    W = W - lrate * dW # 기울기 수정
    b = b - lrate * db # 절편 수정
```



경사 하강법 구현

26

```
# 기울기와 절편을 출력한다.
```

```
print (W, b)
```

```
# 예측값을 만든다.
```

```
y_pred = W*X + b
```

```
# 입력 데이터를 그래프 상에 찍는다.
```

```
plt.scatter(X, y)
```

```
# 예측값은 선그래프로 그린다.
```

```
plt.plot([min(X), max(X)], [min(y_pred), max(y_pred)], color='red')
```

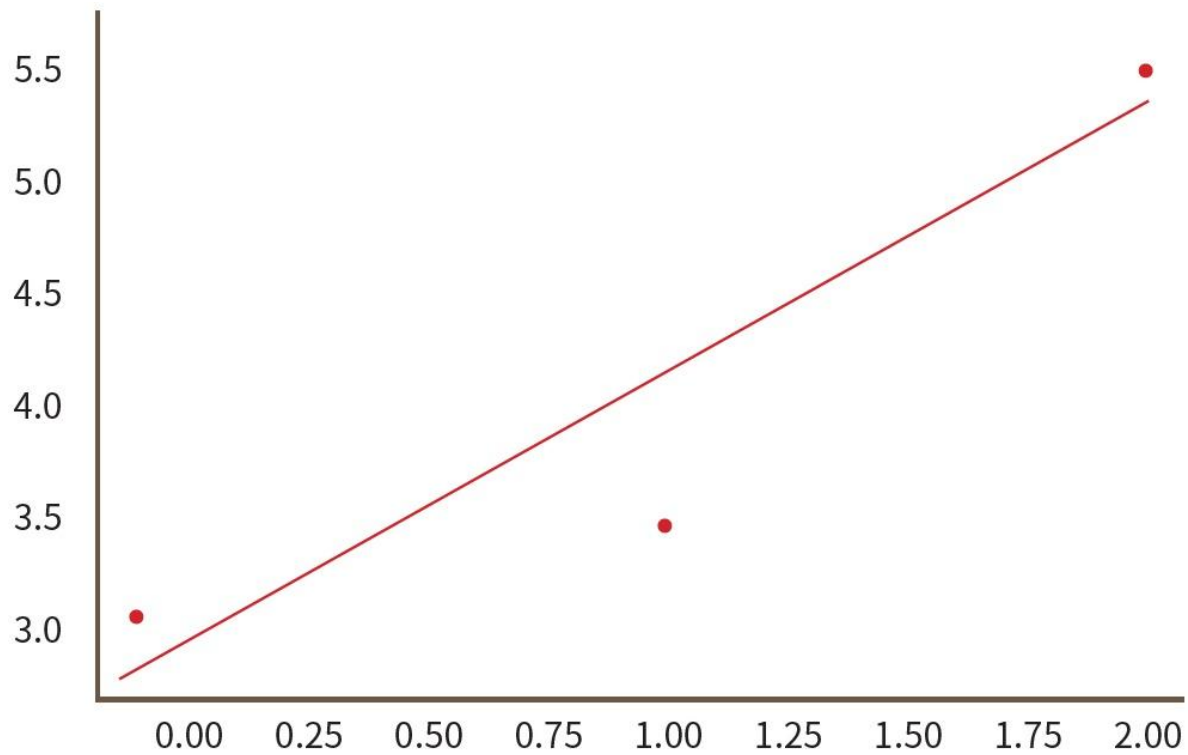
```
plt.show()
```

1.2532418085611319 2.745502230882486



경사 하강법 구현

27





과잉 적합 vs 과소 적합

28

- 과잉 적합(overfitting)이란 훈련 데이터에서는 성능이 뛰어나지만 테스트 데이터(일반화)에 대해서는 성능이 잘 나오지 않는 모델을 생성하는 것이다.
- 과소 적합(underfitting)이란 학습 데이터와 테스트 데이터 모두에서 성능이 좋지 않은 경우이다. 이 경우에는 모델 자체가 적합하지 않은 경우가 많다. 더 나은 모델을 찾아야 한다.



훈련 데이터와 테스트 데이터 (revisited)

29

- 훈련 데이터와 테스트 데이터를 나누는 이유
 - => 학습을 위해 준비한 데이터는 실제 적용되는 경우에 비해 매우 적음
(얼굴인식을 위해 몇 백만장을 준비해도, 전세계 인구 80억명에 비하면 매우 적음)
 - => 훈련 데이터와 테스트 데이터를 나누어 성능 측정을 테스트 데이터로 함으로써
“일반화” 능력을 검증함.
 - => 훈련 데이터에 없는 데이터로 구성된 테스트 데이터로 성능을 측정하여,
실전에 적용되는 데이터에도 비슷한 성능을 나올 것으로 기대하는 것임





훈련 데이터와 테스트 데이터 (revisited)

30

- 좋은 훈련 데이터 & 좋은 테스트 데이터
 - => 특징값 들이 골고루 분포(uniform distribution)하는 것이 좋음
 - => 갖가지 유형을 모두 포함한 연습문제가 좋은 연습 문제이고,
 갖가지 유형을 모두 포함한 시험문제가 좋은 시험 문제임
 - => 현실적으로 이런 사항을 고려해서 만들기가 쉽지 않음
 - => 최대한 데이터를 많이 만들고, 랜덤하게 훈련데이터와 테스트 데이터로
 나누어 학습을 진행하는 것이 일반적임



과잉 적합 vs 과소 적합

31

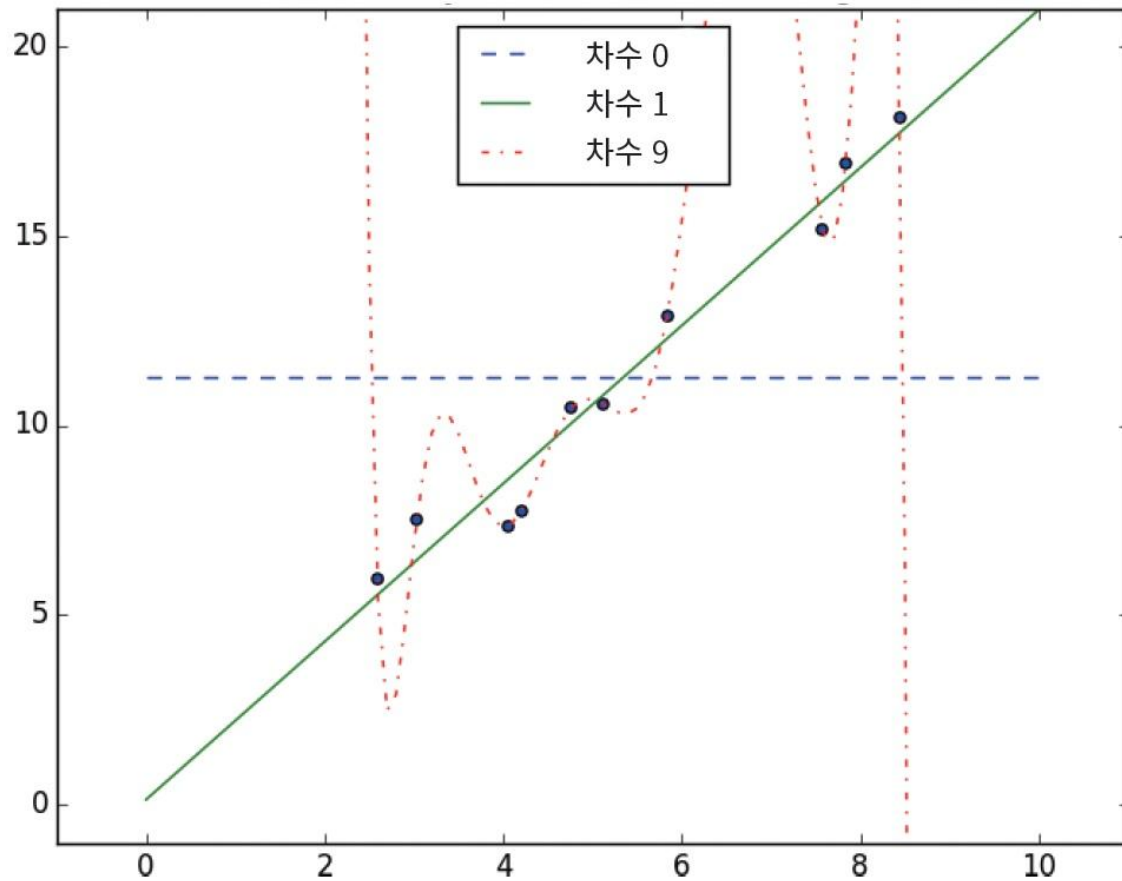


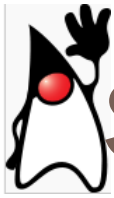
그림 10-5 회귀에서 과잉 적합의 예



과잉 적합 vs 과소 적합

32

훈련 데이터(Training Data)가 매우 적을 경우, 어떤 일이 생길까?



Summary

33

- 지도 학습에는 회귀(regression)와 분류(classification)가 있었다. 전자는 연속적인 값을 예측하고 후자는 입력을 어떤 카테고리 중의 하나로 예측한다.
- 선형 회귀는 입력 데이터를 가장 잘 설명하는 직선의 기울기와 절편 값을 찾는 문제이다.
- 손실 함수(loss function)란 실제 데이터와 직선 간의 차이를 제공한 값이다.
- 회귀란 손실 함수를 최소로 하는 직선의 기울기와 절편값을 계산하는 것이다.
- 손실 함수의 값이 작아지는 방향을 알려면 일반적으로 경사 하강법 (gradient descent method)과 같은 방법을 많이 사용한다.



Q & A

34

